

NEXUS

TECH NEWS

DOCUMENTO DE EVIDENCIAS Y PRUEBAS

Portal de noticias con IA, Docker Compose y balanceo de carga con NGINX

Nombre: JOSUÉ DAVID VELÁSQUEZ IXCHOP

Carné: 202307705

Curso: SOFTWARE AVANZADO

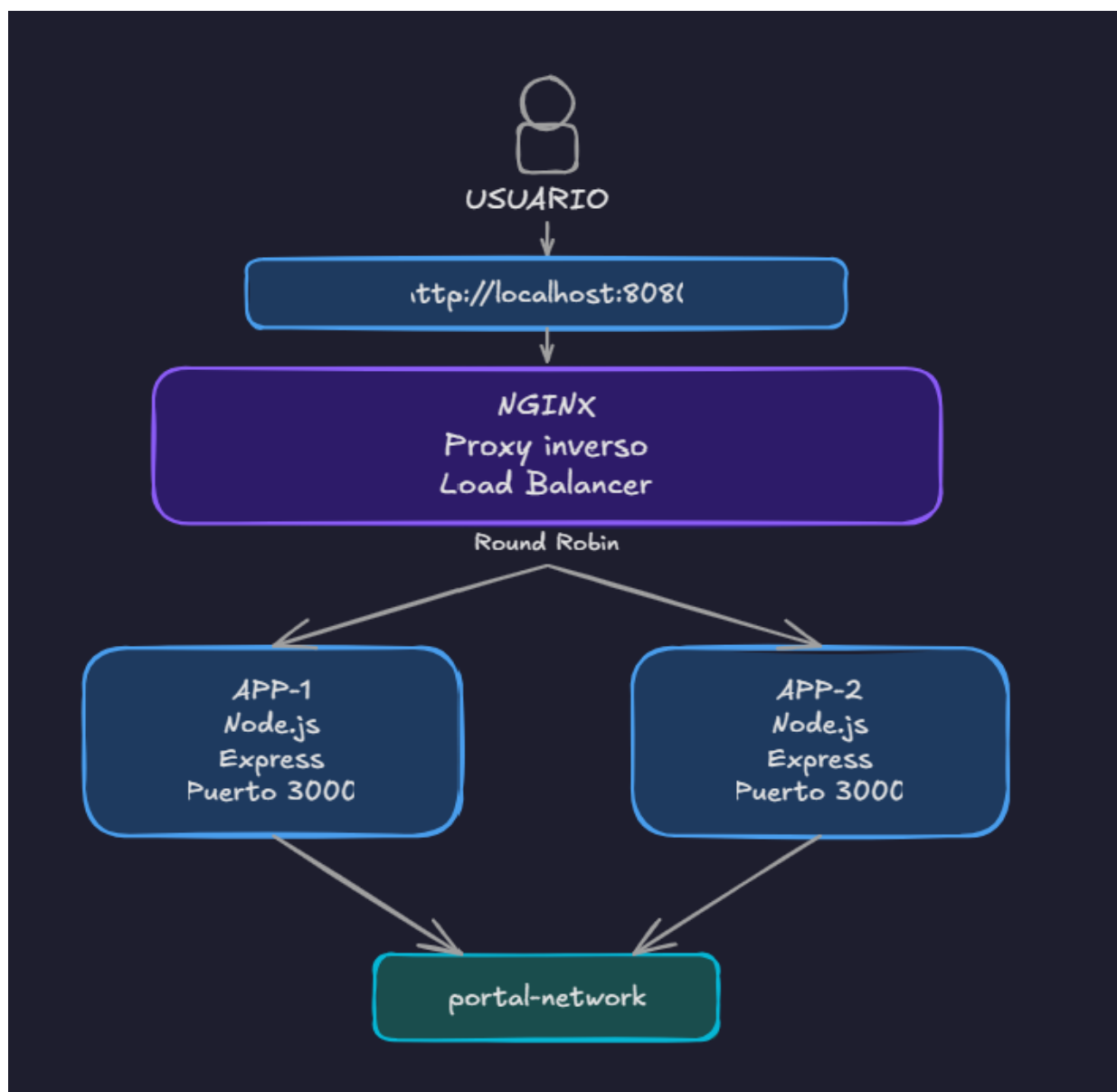
Fecha: 14 DE AGOSTO 2026

1. Introducción

El presente documento reúne las evidencias de funcionamiento del proyecto Nexus Tech News. La solución consiste en un portal de noticias ejecutado en dos instancias de una aplicación Node.js/Express, orquestadas mediante Docker Compose y expuestas únicamente a través de NGINX, que funciona como proxy inverso, enrutador y balanceador de carga mediante el algoritmo Round Robin.

Las pruebas presentadas verifican la creación y ejecución de los contenedores, el acceso mediante NGINX, el funcionamiento de las rutas, la distribución de solicitudes entre dos instancias, la continuidad del servicio al detener una instancia, la identificación de la herramienta de inteligencia artificial y la configuración técnica requerida.

2. Arquitectura evaluada



Punto de entrada: `http://localhost:8080`

Instancias internas: APP-1 y APP-2 en el puerto interno 3000, sin publicación directa al host.

Balanceador: NGINX con grupo upstream y algoritmo Round Robin.

3. Construcción e inicio de la solución desde cero

Objetivo de la evidencia. Demostrar que la solución completa puede construirse e iniciarse mediante Docker Compose utilizando un único comando.

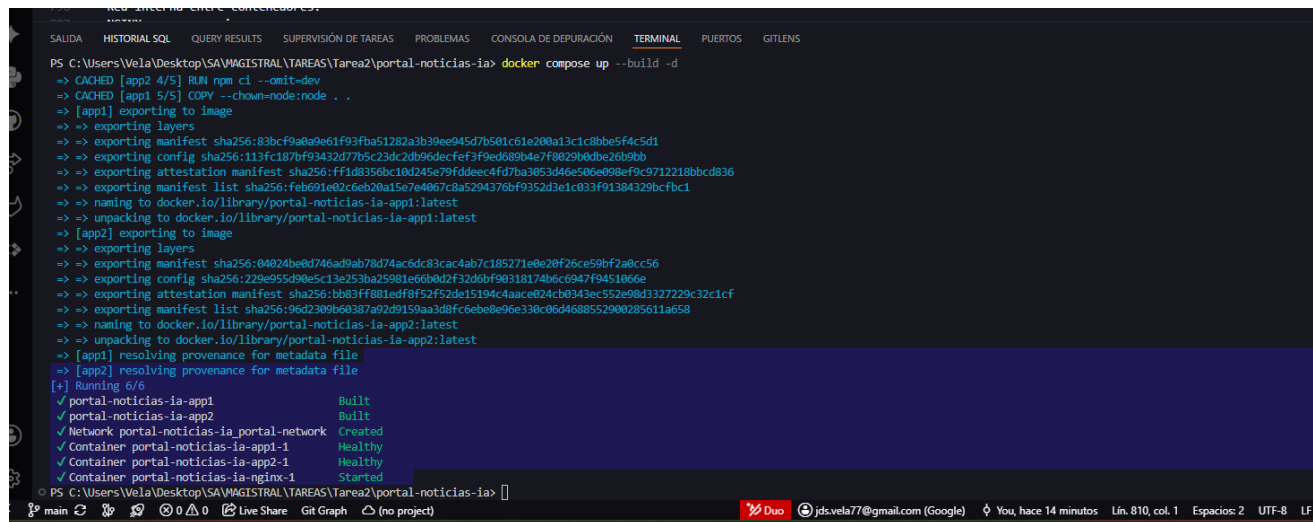
Comando(s) a ejecutar

```
docker compose down
```

```
docker compose up --build -d
```

Qué debe observarse

La terminal debe mostrar la construcción de la imagen de la aplicación y la creación/inicio de los servicios app1, app2 y nginx sin errores.



```
PS C:\Users\Vela\Desktop\SAVWAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> docker compose up --build -d
-> CACHED [app2 4/5] RUN npm ci --omit-dev
-> CACHED [app1 5/5] COPY --chown=node:node .
-> [app1] exporting to image
-> exporting layers
-> exporting manifest sha256:83bcf9a9e61f93fba51282a3b39ee945d7b501c61e200a13c1c8bbe5f4c5d1
-> exporting config sha256:113fc187bf93432d77b5c23dc2db96decfe3f9ed689b4e7f8029b0dbe26b9bb
-> exporting attestation manifest sha256:ff1d8356bc18d245e79fddeec4fd7ba3053d46e506e098ef9c9712218bbcd836
-> exporting manifest list sha256:feb691e02c6eb20a15e7e4067c8a5294376bf9352d3e1c833f91384329bcfbc1
-> naming to docker.io/library/portal-noticias-ia-app1:latest
-> unpacking to docker.io/library/portal-noticias-ia-app1:latest
-> [app2] exporting to image
-> exporting layers
-> exporting manifest sha256:04024be0d746ad9ab78d74ac6dc83cac4ab7c185271e0e20f26ce59bf2a0cc56
-> exporting config sha256:229e955d90e5c13e253ba25981e66b0d2f32d0bf90318174b6c6947f9451066e
-> exporting attestation manifest sha256:bb83ff881edf8f52f52de15194c4aace024cb0343ec552e98d3327229c32c1cf
-> exporting manifest list sha256:96d2309b6087a02d9159aa3d8fceebe8e96e330c06d4688552900285611a658
-> naming to docker.io/library/portal-noticias-ia-app2:latest
-> unpacking to docker.io/library/portal-noticias-ia-app2:latest
-> [app1] resolving provenance for metadata file
-> [app2] resolving provenance for metadata file
[+] Running 6/6
 ✓ portal-noticias-ia-app1      Built
 ✓ portal-noticias-ia-app2      Built
 ✓ Network portal-noticias-ia_portal-network Created
 ✓ Container portal-noticias-ia-app1-1 Healthy
 ✓ Container portal-noticias-ia-app2-1 Healthy
 ✓ Container portal-noticias-ia-nginx-1 Started
PS C:\Users\Vela\Desktop\SAVWAGISTRAL\TAREAS\Tarea2\portal-noticias-ia>
```

Explicación: Demostración que la solución completa puede construirse e iniciarse mediante Docker Compose utilizando un único comando.

4. Estado de los contenedores

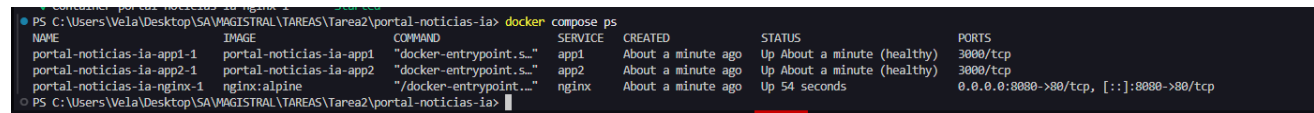
Objetivo de la evidencia. Comprobar que las dos instancias de la aplicación y NGINX se encuentran en ejecución, y que APP-1 y APP-2 alcanzan un estado saludable.

Comando(s) a ejecutar

```
docker compose ps
```

Qué debe observarse

Deben aparecer app1, app2 y nginx. APP-1 y APP-2 deben mostrarse como Up/healthy y NGINX como Up. Solo NGINX debe mostrar la publicación del puerto 8080 hacia el host.



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
portal-noticias-ia-app1-1	portal-noticias-ia-app1	"docker-entrypoint.s..."	app1	About a minute ago	Up About a minute (healthy)	3000/tcp
portal-noticias-ia-app2-1	portal-noticias-ia-app2	"docker-entrypoint.s..."	app2	About a minute ago	Up About a minute (healthy)	3000/tcp
portal-noticias-ia-nginx-1	nginx:alpine	"/docker-entrypoint..."	nginx	About a minute ago	Up 54 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

Explicación: Comprueba que las dos instancias de la aplicación y NGINX se encuentran en ejecución, y que APP-1 y APP-2 alcanzan un estado saludable.

5. Validación de la configuración de NGINX

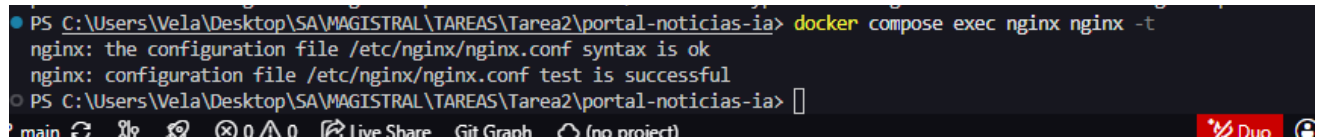
Objetivo de la evidencia. Verificar que el archivo nginx.conf posee una sintaxis válida antes de utilizarlo como proxy inverso y balanceador.

Comando(s) a ejecutar

```
docker compose exec nginx nginx -t
```

Qué debe observarse

NGINX debe indicar que la sintaxis del archivo de configuración es correcta y que la prueba fue exitosa.



```
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> docker compose exec nginx nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia>
```

Explicación: Verificación de que el archivo nginx.conf posee una sintaxis válida antes de utilizarlo como proxy inverso y balanceador.

6. Acceso al portal mediante NGINX

Objetivo de la evidencia. Demostrar que el portal es accesible a través del único punto de entrada publicado por NGINX.

Qué debe observarse

Abrir en el navegador <http://localhost:8080>. Debe mostrarse la página principal de Nexus Tech News con las seis noticias, fotografías, diseño responsive y aviso académico sobre IA.



Explicación : Demostración que el portal es accesible a través del único punto de entrada publicado por NGINX.

7. Funcionamiento de las rutas configuradas

Objetivo de la evidencia. Comprobar que NGINX enruta correctamente la página principal, las noticias individuales y el endpoint de salud.

Comando(s) a ejecutar

```
curl.exe -i http://localhost:8080/health
```

Qué debe observarse

Prueba también en el navegador: <http://localhost:8080/noticias/1> y <http://localhost:8080/noticias/6>. El endpoint /health debe devolver estado ok e identificar una instancia.



Explicación: Comprobación que NGINX enruta correctamente la página principal, las noticias individuales y el endpoint de salud.

8. Identificación del uso de inteligencia artificial

Objetivo de la evidencia. Demostrar que el portal identifica claramente la herramienta de IA utilizada y advierte que el contenido es académico y no periodístico verificado.

Qué debe observarse

En la interfaz debe observarse la advertencia académica y, dentro de las noticias, la atribución “Contenido generado con ChatGPT y revisado para fines académicos”.

herramienta de apoyo y no como un reemplazo del proceso de aprendizaje, análisis y razonamiento necesario para desarrollar software.

Fuente del texto: Contenido generado con ChatGPT y revisado para fines académicos.

Aviso académico: este artículo fue elaborado con fines educativos. El contenido generado o asistido mediante inteligencia artificial no debe considerarse información periodística verificada.

EWS

INS1

Explicación : Demostración que el portal identifica claramente la herramienta de IA utilizada y advierte que el contenido es académico y no periodístico verificado.

9. Prueba de balanceo Round Robin

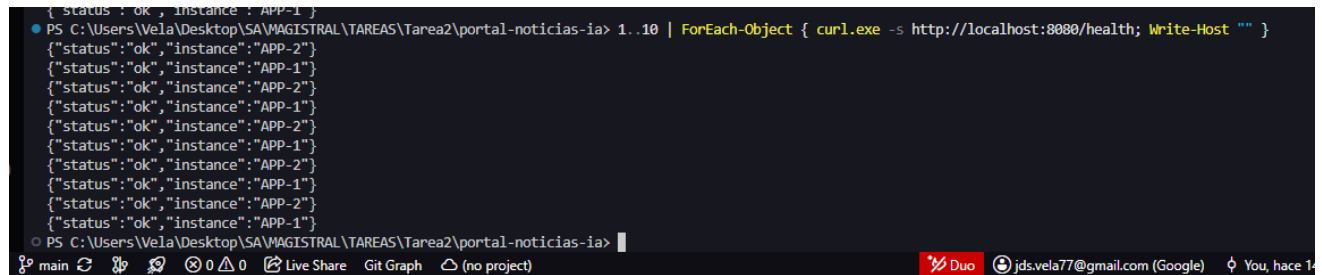
Objetivo de la evidencia. Demostrar que NGINX distribuye solicitudes consecutivas entre APP-1 y APP-2.

Comando(s) a ejecutar

```
1..10 | ForEach-Object { curl.exe -s http://localhost:8080/health; Write-Host "" }
```

Qué debe observarse

Las respuestas deben mostrar ambas instancias, normalmente alternando APP-1 y APP-2. Esta es la evidencia principal del balanceo de carga Round Robin.



```
{ "status": "ok", "instance": "APP-1" }
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> 1..10 | ForEach-Object { curl.exe -s http://localhost:8080/health; Write-Host "" }
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia>
```

Explicación: Demostración que NGINX distribuye solicitudes consecutivas entre APP-1 y APP-2.

10. Verificación de encabezados HTTP

Objetivo de la evidencia. Comprobar técnicamente que la solicitud pasó por NGINX y conocer qué instancia respondió.

Comando(s) a ejecutar

```
curl.exe -i http://localhost:8080/health
```

Qué debe observarse

Entre los encabezados deben aparecer X-Proxy-By: NGINX y X-App-Instance: APP-1 o APP-2. El cuerpo también debe devolver status ok e instance.

```
{ "status": "ok", "instance": "APP-1" }
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> curl.exe -i http://localhost:8080/health
HTTP/1.1 200 OK
Server: nginx/1.31.3
Date: Sun, 09 Aug 2026 22:58:35 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 34
Connection: keep-alive
X-Powered-By: Express
X-App-Instance: APP-2
ETag: W/"22-11AHsBb9+meiqQhuGsKHOf804mg"
X-Proxy-By: NGINX

{"status": "ok", "instance": "APP-2"}
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia>
```

Explicación : Comprobación de que técnicamente la solicitud pasó por NGINX y conocer qué instancia respondió.

11. Detención de la instancia APP-1

Objetivo de la evidencia. Simular la caída de una instancia para comprobar la continuidad del servicio.

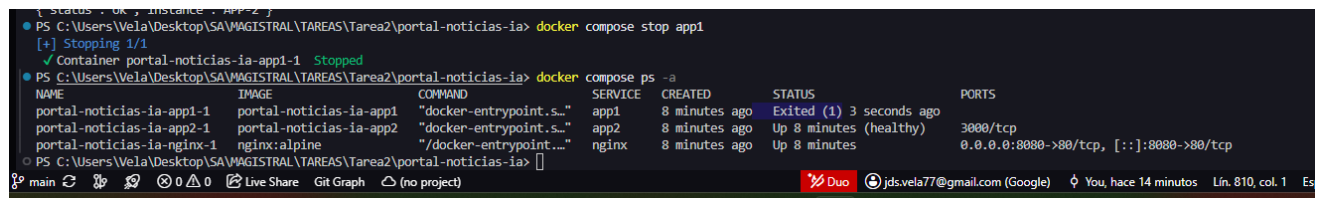
Comando(s) a ejecutar

```
docker compose stop app1
```

```
docker compose ps -a
```

Qué debe observarse

APP-1 debe aparecer detenida/Exited, mientras APP-2 y NGINX permanecen activos.



```
{ status: 'ok', instance: 'APP-2' }
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> docker compose stop app1
[+] Stopping 1/1
✔ Container portal-noticias-ia-app1-1 Stopped
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> docker compose ps -a
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS               PORTS
portal-noticias-ia-app1-1  portal-noticias-ia-app1  "docker-entrypoint.s..."  app1       8 minutes ago  Exited (1) 3 seconds ago  3000/tcp
portal-noticias-ia-app2-1  portal-noticias-ia-app2  "docker-entrypoint.s..."  app2       8 minutes ago  Up 8 minutes (healthy)    3000/tcp
portal-noticias-ia-nginx-1  nginx:alpine          "/docker-entrypoint..."  nginx      8 minutes ago  Up 8 minutes            0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia>
```

Explicación : Simulación de la caída de una instancia para comprobar la continuidad del servicio.

12. Continuidad del servicio con una instancia disponible

Objetivo de la evidencia. Demostrar que el portal continúa respondiendo cuando APP-1 está detenida y APP-2 es la única instancia disponible.

Comando(s) a ejecutar

```
1..8 | ForEach-Object { curl.exe -s http://localhost:8080/health; Write-Host "" }
```

Qué debe observarse

Todas las respuestas deben indicar APP-2. Además, `http://localhost:8080` debe continuar abriendo normalmente en el navegador.

```
PS C:\Users\Veia\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> 1..8 | ForEach-Object { curl.exe -s http://localhost:8080/health; Write-Host "" }
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-2"}
PS C:\Users\Veia\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia>
```

NEXUS TECH NEWS

Portal académico sobre tecnología e inteligencia artificial.

INSTANCIA: APP-2

Contenido generado o asistido con ChatGPT.
Proyecto académico — 2026

Explicación: Demostración que el portal continúa respondiendo cuando APP-1 está detenida y APP-2 es la única instancia disponible.

13. Recuperación de APP-1 y retorno del balanceo

Objetivo de la evidencia. Comprobar que la instancia detenida puede reincorporarse y que NGINX vuelve a distribuir tráfico entre ambas aplicaciones.

Comando(s) a ejecutar

```
docker compose start app1
```

```
docker compose ps
```

```
1..10 | ForEach-Object { curl.exe -s http://localhost:8080/health; Write-Host "" }
```

Qué debe observarse

APP-1 debe volver al estado activo/saludable. En las solicitudes consecutivas deben volver a aparecer APP-1 y APP-2.

```
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> docker compose start app1
[+] Running 1/1
  ✓ Container portal-noticias-ia-app1-1 Started
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
portal-noticias-ia-app1-1  portal-noticias-ia-app1  "docker-entrypoint.s..."  app1       12 minutes ago  Up 37 seconds (healthy)  3000/tcp
portal-noticias-ia-app2-1  portal-noticias-ia-app2  "docker-entrypoint.s..."  app2       12 minutes ago  Up 12 minutes (healthy)  3000/tcp
portal-noticias-ia-nginx-1  nginx:alpine           "/docker-entrypoint..."  nginx      12 minutes ago  Up 12 minutes           0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> 1..10 | ForEach-Object { curl.exe -s http://localhost:8080/health; Write-Host "" }
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
{"status":"ok","instance":"APP-2"}
{"status":"ok","instance":"APP-1"}
{"status":"ok","instance":"APP-2"}
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia>
```

Explicación: Comprobación que la instancia detenida puede reincorporarse y que NGINX vuelve a distribuir tráfico entre ambas aplicaciones.

14. Prueba de acceso exclusivo mediante NGINX

Objetivo de la evidencia. Comprobar que las instancias de aplicación no están publicadas directamente al host y que NGINX es el punto de entrada.

Comando(s) a ejecutar

```
docker compose ps
```

Qué debe observarse

En la columna PORTS, app1 y app2 deben mostrar únicamente el puerto interno 3000/tcp, mientras nginx debe mostrar 0.0.0.0:8080->80/tcp (o equivalente).

```
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
portal-noticias-ia-app1-1	portal-noticias-ia-app1	"docker-entrypoint.s..."	app1	12 minutes ago	Up About a minute (healthy)	3000/tcp
portal-noticias-ia-app2-1	portal-noticias-ia-app2	"docker-entrypoint.s..."	app2	12 minutes ago	Up 12 minutes (healthy)	3000/tcp
portal-noticias-ia-nginx-1	nginx:alpine	"/docker-entrypoint..."	nginx	12 minutes ago	Up 12 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

```
PS C:\Users\Vela\Desktop\SA\MAGISTRAL\TAREAS\Tarea2\portal-noticias-ia>
```

Explicación: Comprobación que las instancias de aplicación no están publicadas directamente al host y que NGINX es el punto de entrada.

15. Diseño adaptable a dispositivos móviles

Objetivo de la evidencia. Demostrar que el portal se adapta a tamaños de pantalla reducidos.

Qué debe observarse

En Chrome abre <http://localhost:8080>, presiona F12 y luego Ctrl + Shift + M. Selecciona un dispositivo móvil. El contenido debe reorganizarse sin desbordamientos y las imágenes deben mantener proporciones adecuadas.



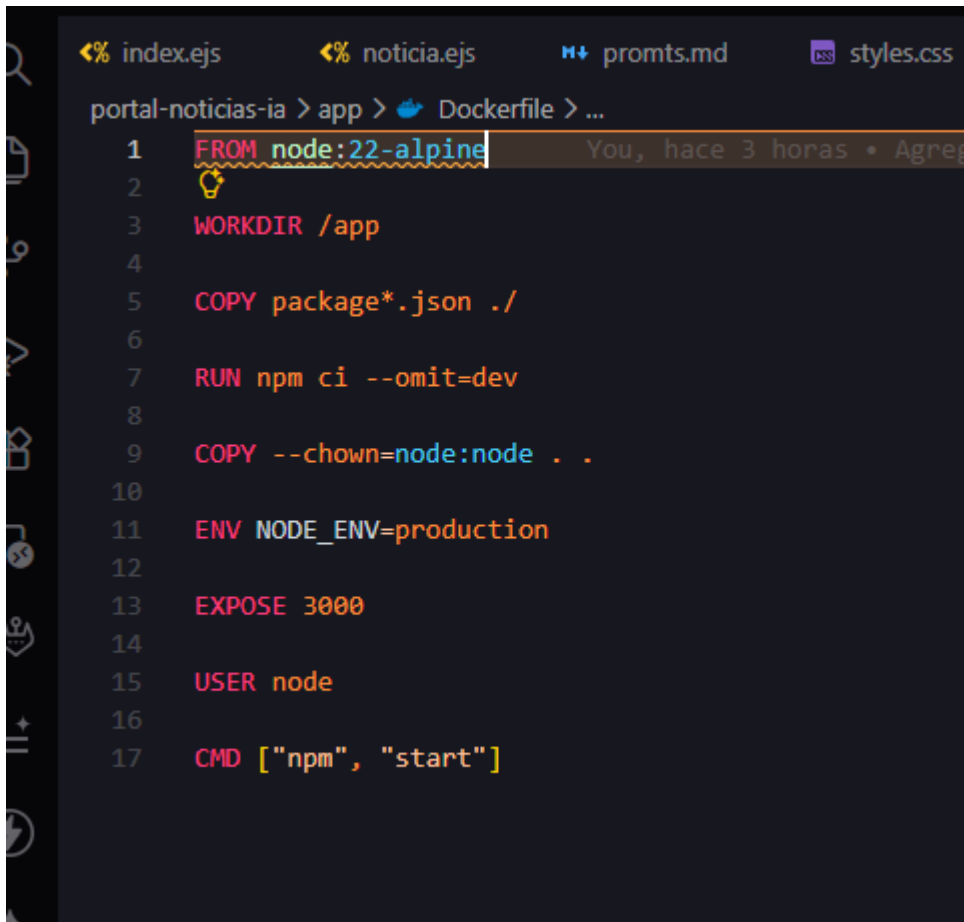
Explicación: Demostración que el portal se adapta a tamaños de pantalla reducidos.

16. Evidencia del Dockerfile

Objetivo de la evidencia. Mostrar el archivo utilizado para construir la imagen Docker de la aplicación.

Qué debe observarse

Abriendo app/Dockerfile en Visual Studio Code. Deben observarse las instrucciones FROM, WORKDIR, COPY, RUN npm ci, ENV, EXPOSE, USER y CMD.

A screenshot of a Visual Studio Code editor showing a Dockerfile. The file explorer on the left shows the path 'portal-noticias-ia > app > Dockerfile'. The editor has tabs for 'index.ejs', 'noticia.ejs', 'promts.md', and 'styles.css'. The Dockerfile content is as follows:

```
1 FROM node:22-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6
7 RUN npm ci --omit=dev
8
9 COPY --chown=node:node . .
10
11 ENV NODE_ENV=production
12
13 EXPOSE 3000
14
15 USER node
16
17 CMD ["npm", "start"]
```

Explicación: Demostración del archivo utilizado para construir la imagen Docker de la aplicación.

17. Evidencia de Docker Compose

Objetivo de la evidencia. Mostrar la configuración que define APP-1, APP-2, NGINX, la red interna, healthchecks y variables de entorno.

Qué debe observarse

Abre docker-compose.yml en Visual Studio Code. Deben verse los servicios app1, app2 y nginx; INSTANCE_NAME; expose; healthcheck; ports 8080:80; depends_on y portal-network.

```
portal-noticias-ia > docker-compose.yml > ...
  Run All Services | You, hace 3 horas | 1 author (You)
1  services:
2
3  app1:
4    build:
5      context: ./app
6      dockerfile: Dockerfile
7
8    environment:
9      PORT: 3000
10     INSTANCE_NAME: APP-1
11
12    expose:
13      - "3000"
14
15    healthcheck:
16      test:
17        [
18          "CMD",
19          "node",
20          "-e",
21          "fetch('http://localhost:3000/health').then(r => { if (!r.ok) process.exit(1); }).catch(() => process.exit(1));"
22        ]
23      interval: 10s
24      timeout: 3s
25      retries: 3
26      start_period: 5s
27
28    networks:
29      - portal-network
30
31    restart: unless-stopped
32
33
34  app2:
35    build:
36      context: ./app
37      dockerfile: Dockerfile
38
39    environment:
40      PORT: 3000
41      INSTANCE_NAME: APP-2
42
43    expose:
44      - "3000"
45
46    healthcheck:
```

```

32
33
34   app2:
35     build:
36       context: ./app
37       dockerfile: Dockerfile
38
39     environment:
40       PORT: 3000
41       INSTANCE_NAME: APP-2
42
43     expose:
44       - "3000"
45
46     healthcheck:
47       test:
48         [
49           "CMD",
50           "node",
51           "-e",
52           "fetch('http://localhost:3000/health').then(r => { if (!r.ok) process.exit(1); }).catch(() => process.exit(1));"
53         ]
54       interval: 10s
55       timeout: 3s
56       retries: 3
57       start_period: 5s
58
59     networks:
60       - portal-network
61
62     restart: unless-stopped
63
64   DRun Service

```

```

60     - portal-network
61
62     restart: unless-stopped
63
64   DRun Service
65   nginx:
66     image: nginx:alpine
67
68     ports:
69       - "8080:80"
70
71     volumes:
72       - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
73
74     depends_on:
75       app1:
76         condition: service_healthy
77
78       app2:
79         condition: service_healthy
80
81     networks:
82       - portal-network
83
84     restart: unless-stopped
85
86
87   networks:
88     portal-network:
89       driver: bridge
90

```

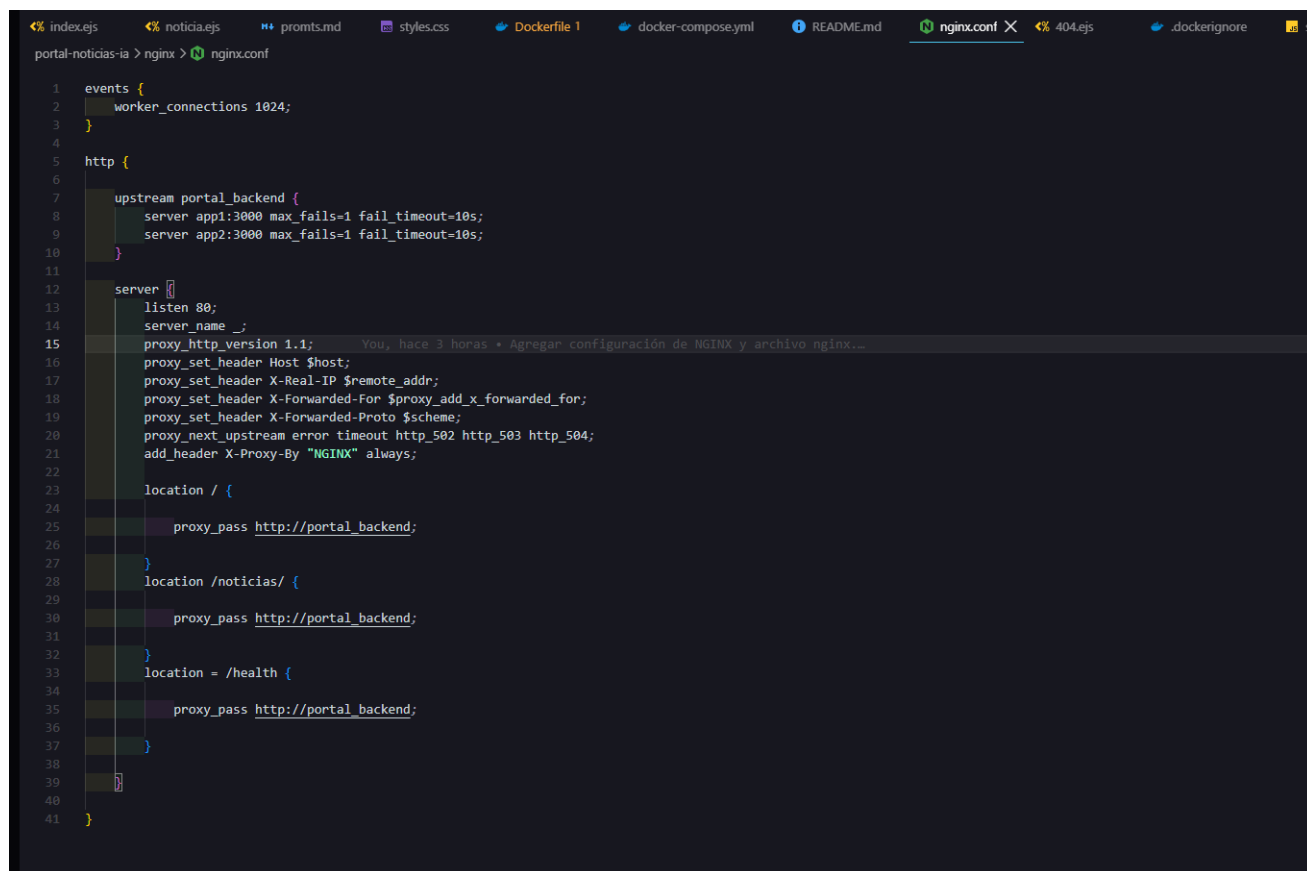
Explicación: Demostración de la configuración que define APP-1, APP-2, NGINX, la red interna, healthchecks y variables de entorno.

18. Evidencia de la configuración de NGINX

Objetivo de la evidencia. Mostrar el grupo upstream, las reglas de enrutamiento, los encabezados y la configuración de continuidad ante fallos.

Qué debe observarse

Abre nginx/nginx.conf en Visual Studio Code. Deben observarse upstream portal_backend, app1:3000, app2:3000, proxy_pass, location /, location /noticias/, location = /health, proxy_set_header y proxy_next_upstream.



```
1  events {
2      worker_connections 1024;
3  }
4
5  http {
6
7      upstream portal_backend {
8          server app1:3000 max_fails=1 fail_timeout=10s;
9          server app2:3000 max_fails=1 fail_timeout=10s;
10     }
11
12     server {
13         listen 80;
14         server_name _;
15         proxy_http_version 1.1;
16         proxy_set_header Host $host;
17         proxy_set_header X-Real-IP $remote_addr;
18         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
19         proxy_set_header X-Forwarded-Proto $scheme;
20         proxy_next_upstream error timeout http_502 http_503 http_504;
21         add_header X-Proxy-By "NGINX" always;
22
23         location / {
24
25             proxy_pass http://portal_backend;
26
27         }
28         location /noticias/ {
29
30             proxy_pass http://portal_backend;
31
32         }
33         location = /health {
34
35             proxy_pass http://portal_backend;
36
37         }
38     }
39
40 }
41 }
```

Explicación: Demostración del grupo upstream, las reglas de enrutamiento, los encabezados y la configuración de continuidad ante fallos.

19. Conclusión de las pruebas

Las pruebas realizadas permiten comprobar que Nexus Tech News se ejecuta correctamente mediante contenedores Docker y que Docker Compose coordina las dos instancias de la aplicación junto con NGINX. El proxy inverso funciona como único punto de entrada y distribuye las solicitudes entre APP-1 y APP-2 mediante Round Robin. Asimismo, se comprobó que el portal continúa disponible cuando una de las instancias se detiene y que, al recuperarla, vuelve a participar en el balanceo de carga.

También se verificó el funcionamiento de las rutas configuradas, la identificación de la instancia mediante la interfaz, el endpoint /health y los encabezados HTTP, así como la atribución obligatoria del contenido generado o asistido mediante ChatGPT. Con ello se cubren las evidencias técnicas solicitadas para el proyecto.